

Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen

Apunte teórico-práctico basado en el contenido de clase y bibliografía oficial

Buses de Datos y de Direcciones

Un microcontrolador (MCU) integra la CPU, la memoria y los periféricos en un solo chip (**System on Chip - SoC**). Para coordinar la transferencia de información entre estos bloques internos se utiliza un conjunto de conductores llamados **buses**.

Concepto de Buses de Interconexión

La CPU se comunica con la memoria RAM, Flash y los periféricos mapeados a través de tres conjuntos principales de señales:

- **Address Bus (Bus de Direcciones):** Unidireccional (proviene de la CPU). Especifica **con qué dispositivo o posición de memoria** se desea comunicar la CPU.
- **Data Bus (Bus de Datos):** Bidireccional. Transporta **el valor real** que se lee o se escribe.
- **Control Bus / Lógica de Control (R/W):** Indica el tipo de operación (Lectura: $R/w = 1$, Escritura: $R/w = 0$).

Bus / Señal	Dirección	Función Principal
Address Bus	CPU → Periférico	Selecciona la posición de memoria o registro interno a acceder.
Data Bus	CPU ↔ Periférico	Transfiere la palabra de datos (8, 16 o 32 bits según la arquitectura).
Control (R/w)	CPU → Periférico	Establece si se lee (1) o si se escribe (0) sobre el bus.

Tabla 1: Resumen de los buses de interconexión interna del microcontrolador.

Buffers Tri-State y Lógica de Decodificación

Dado que múltiples periféricos (RAM, Flash, GPIO, Timers, ADC) comparten el mismo **Data Bus**, se debe evitar que más de un dispositivo intente imponer un nivel lógico simultáneamente, lo cual generaría un cortocircuito (**conflicto de bus**).

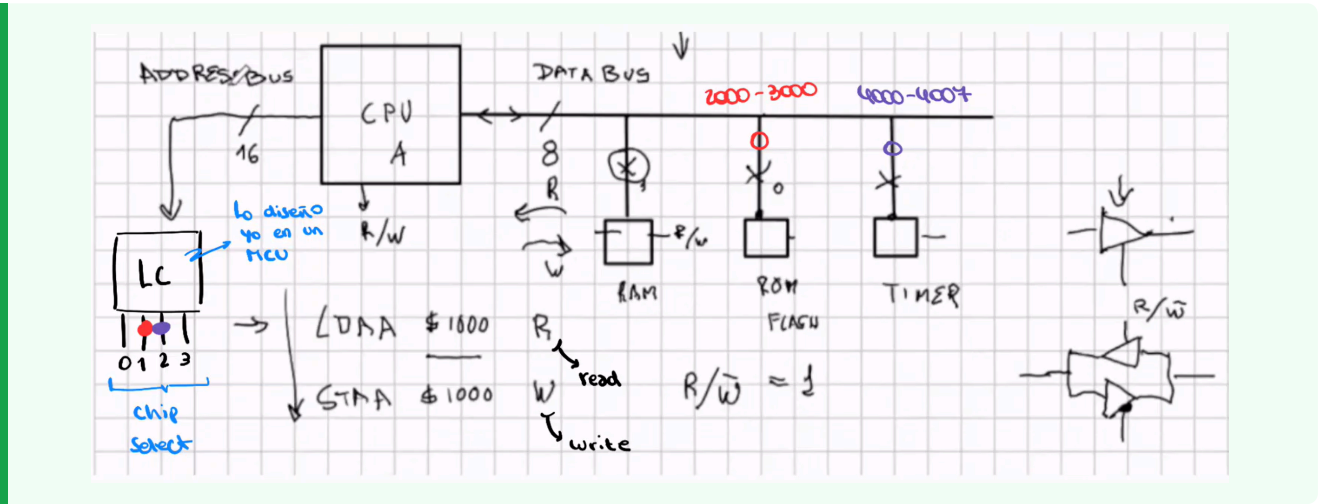
Estados Lógicos de un Buffer Tri-State:

- **Baja Impedancia:** Conectado al bus. Puede imponer nivel 1 o 0.
- **Alta Impedancia (Z):** Desconectado eléctricamente del bus. No interfiere con otros dispositivos.

Decodificador de Direcciones (Address Decoder): Analiza el **Address Bus** y activa la línea **Chip Select (CS)** únicamente del periférico correspondiente al rango de direcciones solicitado.

Ejemplo de Ciclo de Lectura / Escritura

1. **Lectura:** La CPU ejecuta la instrucción LDAA \$1000.
 - Pone la dirección \$1000 en el **Address Bus**.
 - El **Address Decoder** activa el **Chip Select** del bloque correspondiente (ej. RAM).
 - La CPU pone $R/w = 1$.
 - El dispositivo en \$1000 pone su buffer en baja impedancia y coloca el dato en el **Data Bus**. La CPU lo guarda en el acumulador.
2. **Escritura:** La CPU ejecuta STAA \$1000.
 - Pone la dirección \$1000 en el **Address Bus** y $R/w = 0$.
 - Coloca el valor del acumulador en el **Data Bus**. El periférico latché el dato en su registro interno.



GPIO (General Purpose Input/Output)

Un **GPIO** es un periférico modular que permite a la CPU controlar o leer pines físicos del microcontrolador como entradas o salidas digitales.

Mapeo en Memoria (Memory-Mapped I/O)

Los registros que controlan los pines del GPIO están mapeados directamente en el mapa de memoria global del microcontrolador. Escribir o leer un pin digital equivale a realizar un acceso estándar a memoria sobre una dirección específica asignada a ese registro.

Técnica de Readback (Lectura de Salidas)

El **Readback** es la capacidad del hardware de **leer el nivel lógico real** presente en un pin configurado como salida.

¿Para qué sirve el Readback?

- 1. **Verificación de Estado:** Confirmar si una salida físicamente cambió de estado o si existe una sobrecarga/falla en el circuito externo.
- 2. **Operaciones de Modificación:** Permite leer el estado actual antes de realizar un cambio lógico atómico.
- 3. **Diagnóstico:** Un pin de entrada configurado en el mismo nodo puede observar una salida para testear la integridad del circuito.

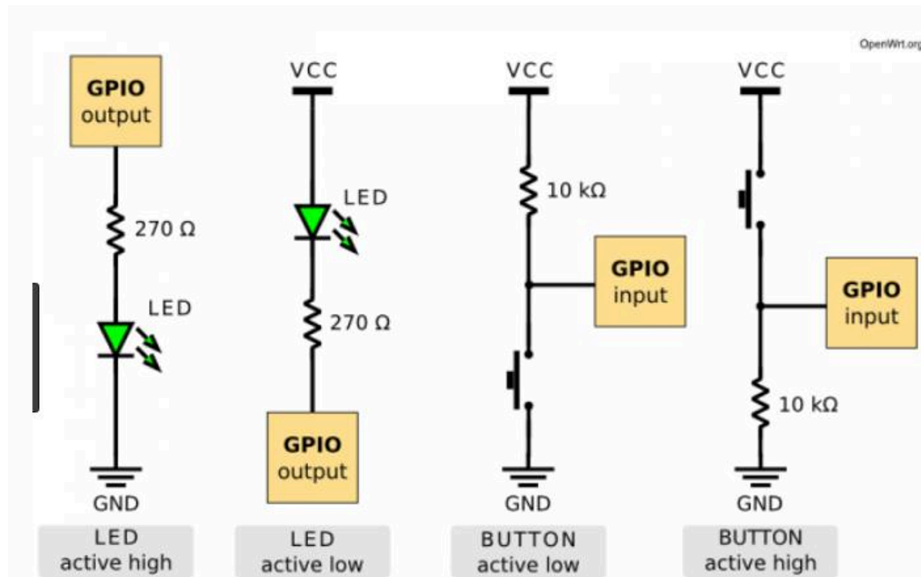
Regla de Seguridad Eléctrica: Nunca se deben configurar dos pines como salidas conectadas al mismo conductor con niveles opuestos (1 y 0), ya que se crea un camino directo a masa produciendo sobrecorriente y daño irreversible.

Lógica Activa: Active-High vs. Active-Low

El nivel lógico (1 ó 0) no siempre coincide con la función “activa” de un componente.

Lógica	Nivel Activo	Simbología	Ejemplo Típico
Active-High	HIGH (V_{DD})	Sin marca o SIG	LED encendido al enviar 1 a GND.
Active-Low	LOW (V_{SS})	Barra SIG, SIG_n, burbuja en diagramas	Botonera con Pull-Up o habilitación LED_EN_n.

Tabla 2: Comparación entre Lógica Activa por Alto y Lógica Activa por Bajo.

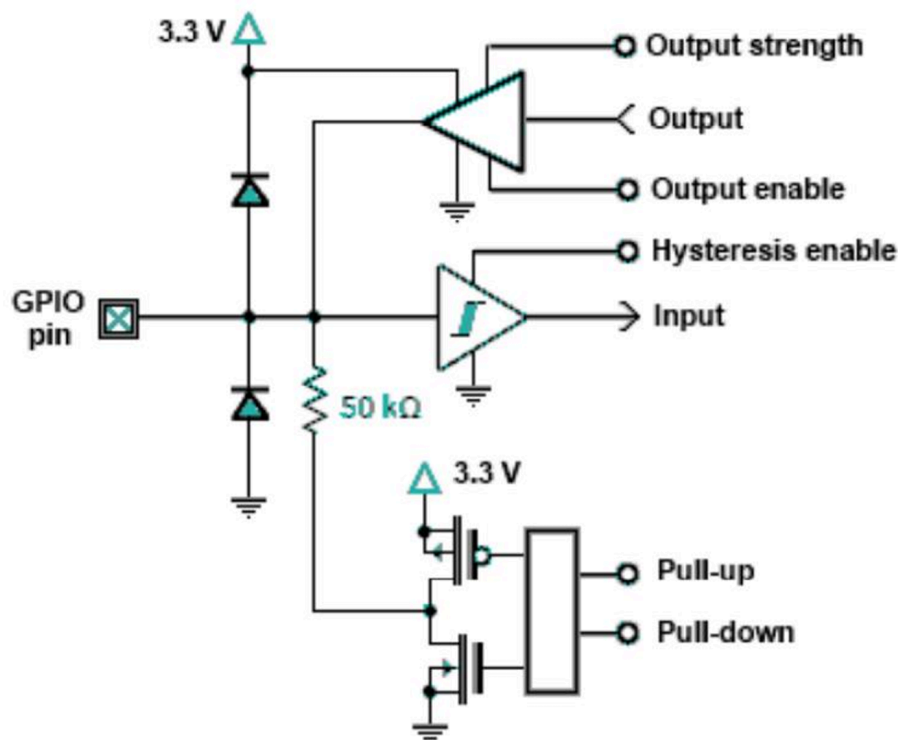


Propiedades Eléctricas de los Pines Digitales

Un pin de I/O no es un simple conductor; contiene circuitos de protección, transistores de conmutación y redes pasivas configurables por software.

Diodos de Protección (ESD/Overvoltage): Dos diodos clamping conectados entre el pin y V_{DD} / V_{SS} . Conducen cuando $V_{pin} > V_{DD} + 0.3V$ o $V_{pin} < -0.3V$. **Atención:** No deben usarse como reguladores permanentes de tensión.

Resistencias de Pull-Up / Pull-Down: Resistencias internas (típicamente 20k Ω – 50k Ω) habilitables por software. Evitan que una entrada quede flotante (estado indeterminado sujeto a ruido electromagnético).



Parámetros de Configuración Dinámica de Salida

1. Drive Strength (Capacidad de Corriente - DSE):

- **Low Drive Strength:** Limita la corriente máxima de salida (ej. 2 mA a 4 mA). Reduce el ruido y el consumo eléctrico.
- **High Drive Strength:** Entrega mayor corriente (ej. 18 mA a 20 mA) para manejar cargas mayores (LEDs, buzzers) directamente sin deformar la señal.

2. Slew Rate Control (Velocidad de Transición - SRE):

- **Slew Rate Enable:** Limita la velocidad de cambio dV/dt de la transición lógico-eléctrica.
- **Ventaja:** Atenúa picos de alta frecuencia y reduce la Interferencia Electromagnética (EMI) y la diafonía en el PCB.

Salidas Open-Drain / Drenador Abierto

En la configuración **Open-Drain**, el transistor MOSFET P superior se deshabilita. El pin solo puede conmutar entre **Masa (0)** y **Alta Impedancia (Z)**.

Aplicaciones de Open-Drain

- **Lógica Colector/Drenador Abierto (Wired-OR):** Permite conectar múltiples salidas al mismo bus (ej. I2C) con un único resistor de Pull-Up externo sin producir cortocircuitos.
- **Traducción de Niveles de Tensión (Level Shifting):** Permite interfaz con circuitos de mayor voltaje (ej. controlar una línea de 5V alimentando el Pull-Up a 5V desde un microcontrolador de 3.3V).

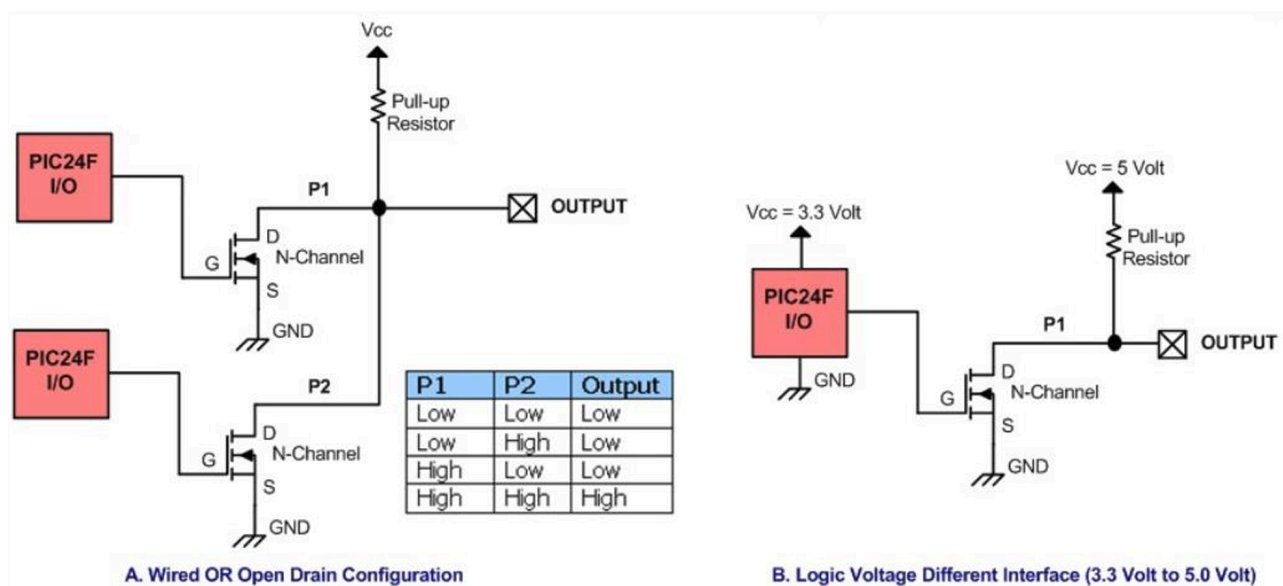


Figura 5: Implementación de lógica digital con MOSFET y elevación de la tensión de salida

El Módulo GPIO en el Microcontrolador NXP Kinetis K64F

El microcontrolador MK64FN1M0VLL12 (ARM Cortex-M4 a 120 Mhz) posee **5 puertos paralelos de 32 bits:** PORTA, PORTB, PORTC, PORTD y PORTE.

Registros de Datos CMSIS (Capa MCAL)

Para operar los datos del GPIO, el SDK provee punteros a la estructura `GPIO_Type` (PTA, PTB, PTC, PTD, PTE). Cada puerto cuenta con registros de 32 bits dedicados (Kinetis K64 - Reference Manual - Chapter 4):

GPIOx_PDDR

Port Data Direction Register (Offset: 0x14)

Define la dirección del pin.

- Bit en 0: Pin configurado como **Entrada Digital (INPUT)**.
- Bit en 1: Pin configurado como **Salida Digital (OUTPUT)**.

GPIOx_PSOR

Offset: 0x04

Port Set Output Register: Escribir un 1 en el bit n pone la salida en **HIGH (1)**. Escribir 0 no produce ningún efecto. Operación atómica sin modificar otros pines.

GPIOx_PCOR

Offset: 0x08

Port Clear Output Register: Escribir un 1 en el bit n fuerza la salida a **LOW (0)**. Escribir 0 no produce ningún efecto. Operación atómica.

GPIOx_PTOR

Offset: 0x0C

Port Toggle Output Register: Escribir un 1 en el bit n **invierte el estado** de la salida (0 $\rightarrow$ 1 ó 1 $\rightarrow$ 0). Operación atómica.

GPIOx_PDIR

Offset: 0x10

Port Data Input Register: Registro de sólo lectura (`__I volatile const`). Retorna el valor lógico real capturado en los pines del puerto.

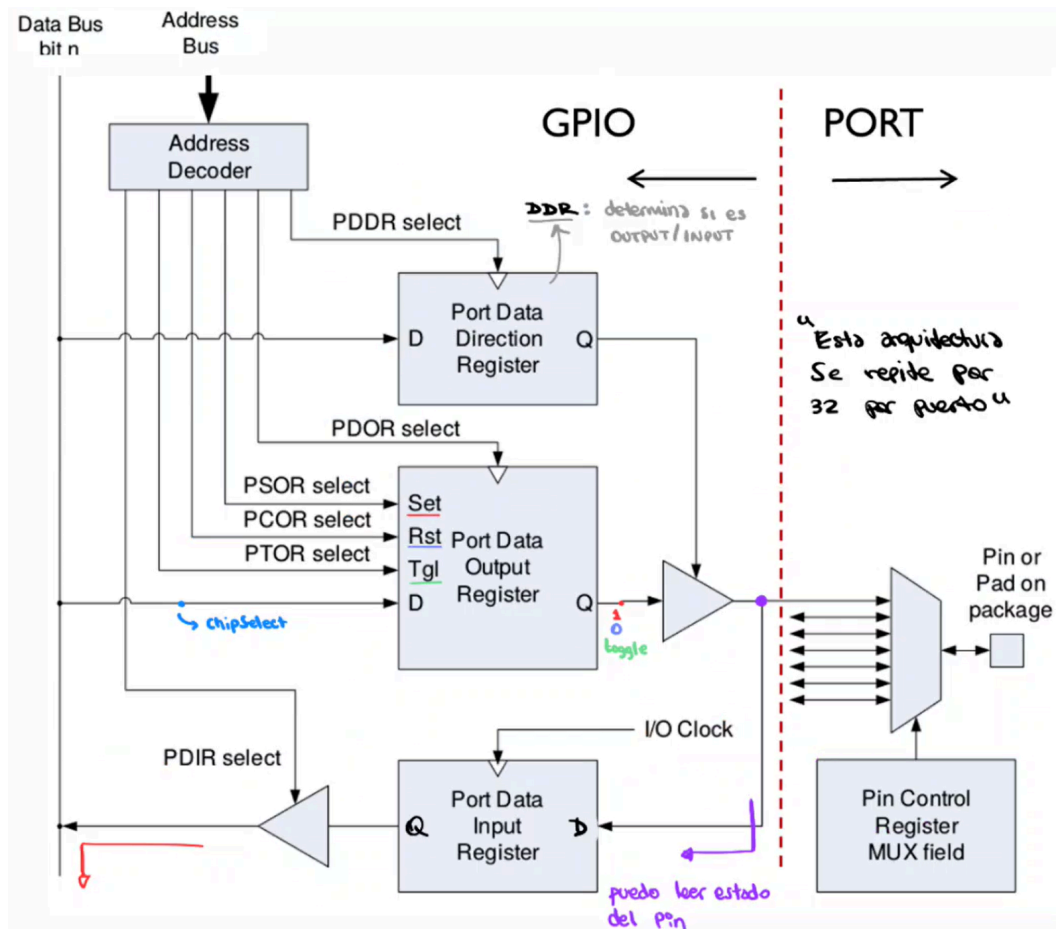


Figura 6: Registros de datos de un puerto GPIO

Ventaja de PSOR / PCOR / PTOR sobre Read-Modify-Write

En microcontroladores tradicionales de 8 bits, para cambiar un solo bit se requería leer todo el puerto, hacer una máscara OR/AND y reescribir (Read-Modify-Write). Esto consume múltiples ciclos de instrucción y **no es atómico** (es susceptible a interrupciones intermedias). En el K64F, escribir en PSOR, PCOR o PTOR altera **exclusivamente el bit deseado en un único ciclo de reloj de bus**.

Pin Control Register (PCR) y Configuración Individual

Cada pin físico de cada puerto posee su propio registro de control individual de 32 bits llamado **PORTX_PCRn** (donde $X \in \{A, B, C, D, E\}$ y $n \in \{0...31\}$).

Campos Clave del Registro PORTx_PCRn

Campo	Bits	Descripción y Valores
ISF	24	Interrupt Status Flag: Se pone en 1 cuando ocurre el evento de interrupción configurado. Se limpia escribiendo un 1 (w1c).
IRQC	19:16	Interrupt Configuration: Selecciona el modo de interrupción/DMA (0000 = Deshabilitado; 1001 = Flanco de Subida; 1010 = Flanco de Bajada; 1011 = Ambos Flancos; 1100 = Nivel Lógico 0).
LK	15	Lock Register: Si se escribe 1, bloquea los bits [15 : 0] del PCR hasta el próximo Reset del microcontrolador.
MUX	10:8	Pin Mux Control: Selecciona qué periférico interno se conecta al pin físico. Ver tabla MUX abajo.
DSE	6	Drive Strength Enable: 0 = Baja capacidad de corriente; 1 = Alta capacidad de corriente.
SRE	2	Slew Rate Enable: 0 = Rápido (Fast); 1 = Lento/Controlado (Slow).
PE	1	Pull Enable: 0 = Resistencia de Pull deshabilitada; 1 = Habilitada.
PS	0	Pull Select: 0 = Internal Pull-Down ; 1 = Internal Pull-Up (requiere PE = 1).

Tabla 3: Distribución de campos en el Pin Control Register (PCR).

Multiplexación de Pines (Campo MUX)

Valor MUX	Función del Pin
000 (ALT0)	Pin Deshabilitado / Entrada Analógica (ADC/CMP)
001 (ALT1)	GPIO (General Purpose I/O)
010 (ALT2)	Periférico dedicado 1 (ej. UART, SPI)
011 (ALT3)	Periférico dedicado 2 (ej. FTM / Timer)
100..111	Otras alternativas del Datasheet (ej. Trace, I2S)

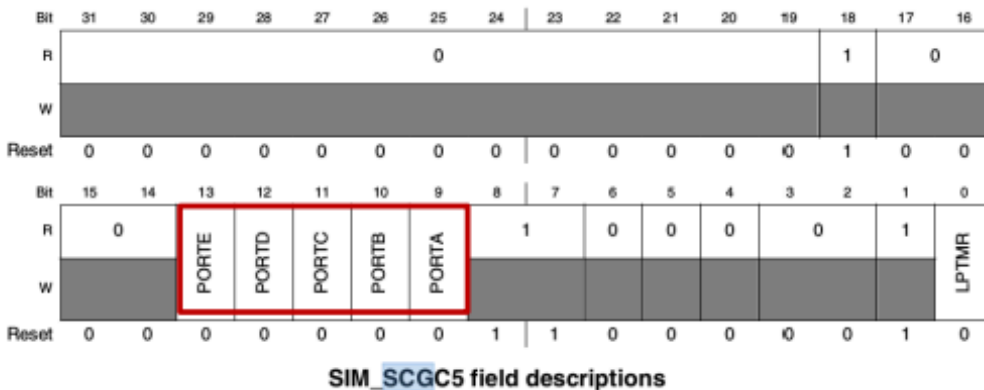
Registros Globales de PCR: Para evitar configurar los 32 pines uno por uno en código redundante:

- **PORTx_GPCLR:** Escribe la misma configuración en los bits [15 : 0] del PCR de múltiples pines.
- **PORTx_GPCHR:** Idem para los bits [31 : 16].

Secuencia Paso a Paso para Configurar un Pin como GPIO

REGLA DE ORO: System Clock Gating Control (SCGC)

Por defecto, tras un Reset, **los relojes de todos los puertos están deshabilitados** para minimizar el consumo. Intenta leer o escribir cualquier registro de un puerto (PCR o GPIO) sin haber habilitado primero su reloj en SIM->SCGC5 provocará una excepción de hardware inmediata (**HardFault**).



Port X Clock Gate control

This bit controls the clock gate to the port X module

0 Clock disabled

1 Clock enabled

X= A,B,C,D,E

Algoritmo de Inicialización Estándar

1. **Habilitar el Clock del Puerto:** Setear el bit correspondiente en SIM->SCGC5 (ej. SIM_SCGC5_PORTB_MASK).
2. **Configurar el Registro PCR (PORTx_PCRn):**
 - Asignar MUX = 1 (ALT1) para operar como GPIO.
 - Configurar resistores de Pull-Up/Pull-Down, Drive Strength y Slew Rate según la necesidad eléctrica del circuito.
3. **Definir Dirección en GPIOx_PDDR:**
 - Setear bit en 0 para Entrada (INPUT).
 - Setear bit en 1 para Salida (OUTPUT).
4. **Operación de Lectura / Escritura:** Operar sobre los registros de datos (PSOR, PCOR, PTOR, PDIR).

Ejemplo en Código C (Capas MCAL y Driver)

El siguiente código ilustra la inicialización y uso de un LED (Salida en PTB21) y un Pulsador con Pull-Up (Entrada en PTA4):

```
#include "MK64F12.h"
#include <stdbool.h>
```

```
void App_Init(void) {
    // 1. GATING: Habilitar relojes de los puertos A y B en la SIM. Operacion OR
    SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK;

    // 2. CONFIGURACIÓN DE SALIDA (LED Rojo en PTB21)
    // Limpiar PCR y seleccionar MUX = ALT1 (GPIO) + High Drive Strength
    PORTB->PCR[21] = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;

    // Configurar dirección como Salida en PDDR
    PTB->PDDR |= (1 << 21);

    // Estado inicial: Apagar LED (Supóngase Active-Low, se envía HIGH con PSOR)
    PTB->PSOR = (1 << 21);

    // 3. CONFIGURACIÓN DE ENTRADA (Pulsador SW3 en PTA4)
    // MUX = ALT1 (GPIO) + Habilitar Pull-Up interno (PE=1, PS=1)
    PORTA->PCR[4] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;

    // Configurar dirección como Entrada en PDDR (bit en 0)
    PTA->PDDR &= ~(1 << 4);
}

void App_Run(void) {
    // Leer estado del pulsador desde PDIR
    bool sw3_presionado = !(PTA->PDIR & (1 << 4)); // Active-Low

    if (sw3_presionado) {
        // Si se presiona, encender LED (forzar a LOW con PCOR)
        PTB->PCOR = (1 << 21);
    } else {
        // De lo contrario, apagar LED (forzar a HIGH con PSOR)
        PTB->PSOR = (1 << 21);
    }
}
```

1. Bit-Banding: Operaciones Atómicas de BITS

En los microcontroladores estándar de 32 bits, la CPU **no puede modificar bits individuales directamente en memoria o en un registro**. Los accesos a memoria se realizan siempre por bytes (8 bits), half-words (16 bits) o words completos (32 bits).

El Problema de la Condición de Carrera (Race Condition)

Cuando intentamos cambiar un solo bit mediante la secuencia tradicional **Read-Modify-Write (RMW)**:

1. **Read:** La CPU lee el registro completo de 32 bits a un registro interno de la CPU.
2. **Modify:** Realiza una operación lógica (OR, AND) para alterar el bit deseado.
3. **Write:** Escribe el valor modificado de regreso al registro.

⚠ Riesgo de Concurrencia con Interrupciones

Si una interrupción (ISR) ocurre justo entre el paso de **Lectura** y el de **Escritura**, la ISR puede modificar otro bit del mismo registro. Al retornar de la interrupción, el programa principal ejecutará el paso de **Escritura**, sobrescribiendo y perdiendo completamente el cambio realizado por la ISR.

La Solución: Mapeo Bit-Band

La arquitectura ARM Cortex-M4 introduce el mecanismo de Bit-Banding, el cual convierte un acceso a un **bit individual** en una operación de escritura o lectura **atómica** (indivisible e ininterrumpible) mediante el uso de un alias de memoria.

💡 Región Bit-Band

- Corresponde a la región de memoria real (SRAM o Periféricos).
- Cada bit en esta región se mapea a una palabra de 32 bits en la región alias.
- Tamaño: 1 MB de memoria real.

💡 Región Alias Bit-Band

- Dirección especial de memoria de 32 MB.
- Escribir 0x01 o 0x00 en una dirección alias modifica solamente el bit objetivo de manera atómica por hardware.

Fórmula de Mapeo de Direcciones Para calcular la dirección en la región alias correspondiente a un bit específico de un byte/registro:

$$\text{Alias_Addr} = \text{Alias_Base} + (\text{Byte_Offset} \times 32) + (\text{Bit_Number} \times 4)$$

Zona de Memoria	Dirección Base Real (Bit-Band)	Dirección Base Alias
SRAM (RAM)	0x2000_0000	0x2200_0000
Periféricos (GPIO, TIM, etc.)	0x4000_0000	0x4200_0000

🔧 Lectura / Escritura en Alias

- **Escritura:** Escribir cualquier número con LSB=1 (como 0x01) en la dirección alias pone el bit real en 1. Escribir un número con LSB=0 (como 0x00) pone el bit real en 0.
- **Lectura:** Leer la dirección alias retorna 0x01 si el bit real está en 1, y 0x00 si está en 0.

2. Fundamentos de Interrupciones

Una interrupción es un evento generado por el hardware que suspende temporalmente la ejecución secuencial del programa principal (**Main Loop**) para atender una tarea urgente mediante una rutina llamada ISR (**Interrupt Service Routine**). Una vez finalizada la ISR, el procesador retoma la ejecución del programa exactamente en la instrucción donde fue suspendido.

Conceptos Fundamentales

- **IRQ (Interrupt Request):** Solicitud asincrónica de interrupción enviada por un periférico al procesador.
- **ISR (Interrupt Service Routine / Handler):** Función ejecutada en respuesta a una IRQ específica.
- **IVT (Interrupt Vector Table):** Tabla ubicada al inicio de la memoria que almacena punteros a función (**vectores**) con las direcciones de inicio de cada ISR.

Comparativa de Estrategias de Control

Mecanismo	Funcionamiento	Ventajas	Desventajas
Polling (Consulta)	El bucle principal consulta continuamente el estado del periférico en un <code>while(1)</code> .	Fácil de programar, sin sobrecosto de contexto.	Desperdicia tiempo de CPU y energía; no determinístico.
Interrupción Periódica (Tick)	Un timer (SysTick) interrumpe a intervalos regulares para muestrear eventos (p. ej. botones).	Consumo controlado, excelente para debouncing de pulsadores.	Retardo de respuesta acotado por el período del tick.
Interrupción Dedicada	El evento de hardware exterior (flanco en pin) dispara la ISR en tiempo real.	Respuesta inmediata (mínima latencia), ultra eficiente.	Puede saturar la CPU si ocurren eventos indeseados a muy alta frecuencia.

Máscaras y Flags de Interrupción

Para que una interrupción sea atendida, se requieren dos llaves en serie:



1. Máscara Global

- Controlada por el procesador (**Global Interrupt Enable** / bit `I` en `PRIMASK`).
- Permite habilitar o deshabilitar todas las interrupciones enmascarables de forma centralizada (funciones `hw_EnableInterrupts()` / `hw_DisableInterrupts()`).



2. Máscaras Individuales

- Nivel Periférico: Bit `IRQC` en el PCR del pin.
- Nivel NVIC: Registros `ISER` / `ICER` del controlador de interrupciones del ARM Cortex-M4.



Flags de Interrupción y Lectura/Limpieza (`w1c`)

Cuando ocurre el evento, el hardware activa un bit de estado llamado Interrupt Status Flag (ej. en `ISFR`).

Regla de Oro: En la mayoría de las ISRs de periféricos (como los puertos GPIO), es obligatorio limpiar el flag dentro de la rutina escribiendo un 1 (Write-1-to-Clear / w1c). Si no se limpia el flag, al salir de la ISR el procesador verá el flag encendido y volverá a entrar a la interrupción en un bucle infinito.

3. El Controlador de Interrupciones del K64: NVIC

El NVIC (**Nested Vectored Interrupt Controller**) es el módulo integrado en el núcleo ARM Cortex-M4 que administra todas las excepciones e interrupciones del microcontrolador.

Características del NVIC

1. Vectoreado: Determina la dirección de la ISR directamente mediante la tabla IVT.
2. Anidado (Nested): Si se dispara una interrupción de **mayor prioridad** mientras se ejecuta otra ISR de menor prioridad, la primera es suspendida para atender a la más urgente.
3. Tail-Chaining: Optimización por hardware que reduce el tiempo de cambio entre dos interrupciones consecutivas a solo 6 ciclos de reloj (evita desapilar y volver a apilar registros innecesariamente).

Mapa de Excepciones e Interrupciones

Las excepciones se dividen en internas (del núcleo ARM Cortex) y externas (periféricos del fabricante NXP):

Nº Excepción	Nombre	Prioridad	Descripción
1	Reset	-3 (Máxima)	Reinicio del sistema por hardware o software.
2	NMI	-2	Interrupción No Enmascarable (urgencias graves).
3	HardFault	-1	Falla grave de hardware (ej. acceso no permitido a memoria).
15	SysTick	Configurable	Timer de sistema periódico propio del núcleo ARM.
16 (IRQ0)	DMA / UART / PORTA...	Configurable	Interrupciones externas de periféricos del K64F.

Estructura de Registros del NVIC (NVIC_Type en CMSIS)

El NVIC se programa mediante una estructura de registros mapeados en memoria:

Registros Principales del NVIC

- ISER[8] (Interrupt Set-Enable): Escribir 1 habilita la línea de interrupción individual.
- ICER[8] (Interrupt Clear-Enable): Escribir 1 deshabilita la línea de interrupción.
- ISPR[8] (Interrupt Set-Pending): Fuerza la interrupción a estado pendiente por software.
- ICPR[8] (Interrupt Clear-Pending): Limpia el estado pendiente.
- IABR[8] (Interrupt Active Bit): Indica si una ISR está actualmente en ejecución.
- IP[240] (Interrupt Priority): Array de bytes para configurar la prioridad. En el K64F se utilizan solo los 4 bits más significativos de cada byte ($2^4 = 16$ niveles de prioridad, donde 0 es la máxima prioridad y 15 la mínima).

Funciones Estándar CMSIS para el NVIC

```
// Habilitar / Deshabilitar interrupción de un periférico
NVIC_EnableIRQ(PORTA_IRQn);
NVIC_DisableIRQ(PORTA_IRQn);

// Configurar prioridad (IRQn_Type IRQn, uint32_t priority [0..15])
NVIC_SetPriority(PORTA_IRQn, 5);

// Limpiar estado pendiente
NVIC_ClearPendingIRQ(PORTA_IRQn);
```

4. Guía Paso a Paso: Configuración de Interrupción en C

Para configurar e implementar una interrupción de GPIO externa (por ejemplo, en el pulsador SW3 conectado a PTA4), se debe seguir estrictamente la siguiente secuencia de 5 pasos:



Secuencia de Inicialización de una Interrupción (5 Pasos Globales)

1. Deshabilitar Interrupciones Globales: Evita que entren interrupciones mientras se configura el microcontrolador (`hw_DisableInterrupts()`).
2. Clock Gating: Activar el reloj del puerto correspondiente (`SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK`).
3. Configurar Pin Control Register (PCR): Configurar MUX como GPIO (ALT1), resistencias de Pull-Up/Down y el modo de disparo por flanco en IRQC.
4. Habilitar IRQ en el NVIC: Activar la atención de interrupción del puerto en el controlador del núcleo (`NVIC_EnableIRQ(PORTA_IRQn)`).
5. Habilitar Interrupciones Globales: Permitir la entrada de interrupciones al procesador (`hw_EnableInterrupts()`).

Código de Ejemplo Completo (Pulsador SW3 en PTA4 e ISR)

```
#include "MK64F12.h"

// 1. Definición de la Rutina de Servicio de Interrupción (ISR)
void PORTA_IRQHandler(void) {
    // A) Verificar e identificar el pin que interrumpió
    if (PORTA->ISFR & (1 << 4)) {

        // B) LIMPIAR EL FLAG DE INTERRUPCIÓN (Write 1 to Clear)
        PORTA->ISFR = (1 << 4);

        // C) Ejecutar la acción deseada (ejemplo: alternar LED)
        PTB->PTOR = (1 << 21); // Toggle LED Azul en PTB21
    }
}

// 2. Función de Inicialización del Sistema
void App_Init(void) {
    // Paso 1: Deshabilitar interrupciones globales durante la configuración
    hw_DisableInterrupts();

    // Paso 2: Activar Clock Gating para los puertos A y B
    SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK;
```

```

// Configurar LED Azul en PTB21 como salida
PORTB->PCR[21] = PORT_PCR_MUX(1); // GPIO (ALT1)
PTB->PDDR |= (1 << 21);           // Dirección: Salida

// Paso 3: Configurar el Pin Control Register (PCR) del Pulsador PTA4
PORTA->PCR[4] = 0x0;                // Limpiar registro
PORTA->PCR[4] |= PORT_PCR_MUX(1);   // MUX = ALT1 (GPIO)
PORTA->PCR[4] |= PORT_PCR_PE(1) | PORT_PCR_PS(1); // Habilitar Pull-Up interno
PORTA->PCR[4] |= PORT_PCR_IRQC(0x09); // IRQC = 0x09 (Flanco de Subida /
Interrupt on Rising Edge)

// Configurar PTA4 como entrada
PTA->PDDR &= ~(1 << 4);

// Paso 4: Configurar prioridad y habilitar la IRQ del Puerto A en el NVIC
NVIC_SetPriority(PORTA_IRQn, 3); // Prioridad 3
NVIC_EnableIRQ(PORTA_IRQn);      // Habilitar en NVIC

// Paso 5: Re-habilitar interrupciones globales
hw_EnableInterrupts();
}

```

Resumen Visual de Configuración IRQC en PCR

Los bits [19:16] del registro `PORTx_PCRn` determinan el modo de generación de evento de interrupción/DMA:

- 0000 (0x0): Interrupción deshabilitada.
- 1000 (0x8): Interrupción lógica 0 (por nivel).
- 1001 (0x9): Interrupción por Flanco de Subida (Rising Edge).
- 1010 (0x10 / 0xA): Interrupción por Flanco de Bajada (Falling Edge).
- 1011 (0x11 / 0xB): Interrupción por Ambos Flancos (Either Edge).
- 1100 (0x12 / 0xC): Interrupción lógica 1 (por nivel).

Mecanismos Avanzados de Gestión de Prioridades en Cortex-M4

La arquitectura **ARM Cortex-M4** incluye optimizaciones por hardware diseñadas para minimizar la latencia de interrupción y el sobrecosto (**overhead**) del apilado y desapilado de registros en la memoria Stack.

1.1. Tail-Chaining (Encadenamiento al Final)

En procesadores convencionales, al finalizar la ejecución de una ISR, la CPU realiza la restauración completa de registros (**POP**) hacia el programa principal, para luego realizar inmediatamente un guardado de registros (**PUSH**) si existe otra interrupción pendiente.

💡 Mecanismo de Tail-Chaining

- **Funcionamiento:** Si al terminar una ISR existe otra interrupción pendiente con la prioridad necesaria para ser atendida, el hardware **omite la secuencia completa de POP y PUSH**.
- **Ahorro de Ciclos:** En lugar de gastar más de 20 ciclos restaurando y volviendo a salvar el contexto, la CPU salta directamente a la segunda ISR realizando una transición administrativa de solo **6 ciclos de reloj**.
- **Beneficio:** Reduce drásticamente la latencia para atender interrupciones consecutivas acumuladas.

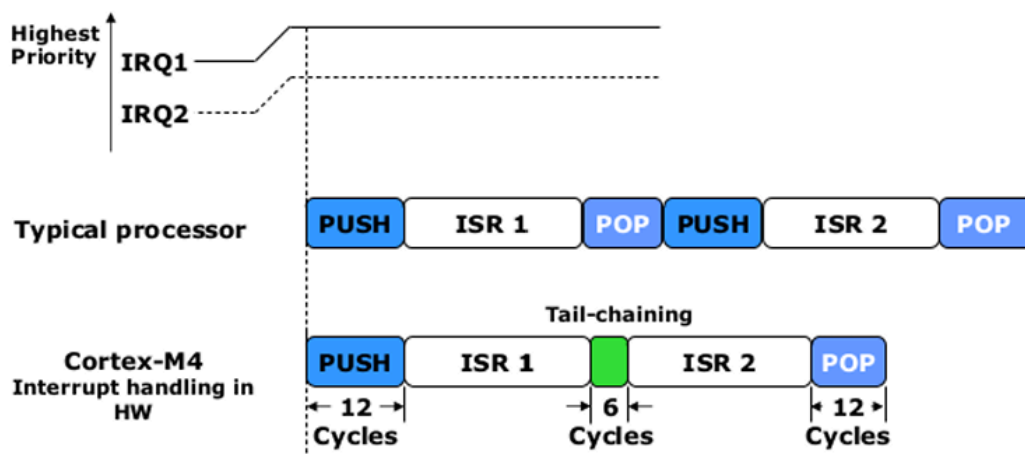


Figura 8: Interrupt Latency - Tail Chaining

1.2. Pre-Emption (Anidamiento y Desalojo de Interrupciones)

El NVIC permite el anidamiento de interrupciones basado en niveles de prioridad.

- **Regla de Desalojo:** Una interrupción entrante solo desaloja (**preempts**) a la ISR que se está ejecutando actualmente si su **nivel de prioridad es strictly mayor** (es decir, posee un número numérico menor en la configuración de prioridad).
- **Prioridad Igual o Menor:** Si la interrupción entrante tiene prioridad igual o más baja que la activa, permanece retenida en estado pendiente (**Pending**) hasta que finalice la ISR en curso, momento en el cual se atiende mediante **Tail-Chaining**.

⚠ Guardado Automático de Contexto

Cuando ocurre el desalojo por mayor prioridad, el hardware apila automáticamente el bloque de registros básicos en la pila (**MSP/PSP**) para permitir la ejecución de la nueva ISR y reanudar la ISR anterior al finalizar la más urgente.

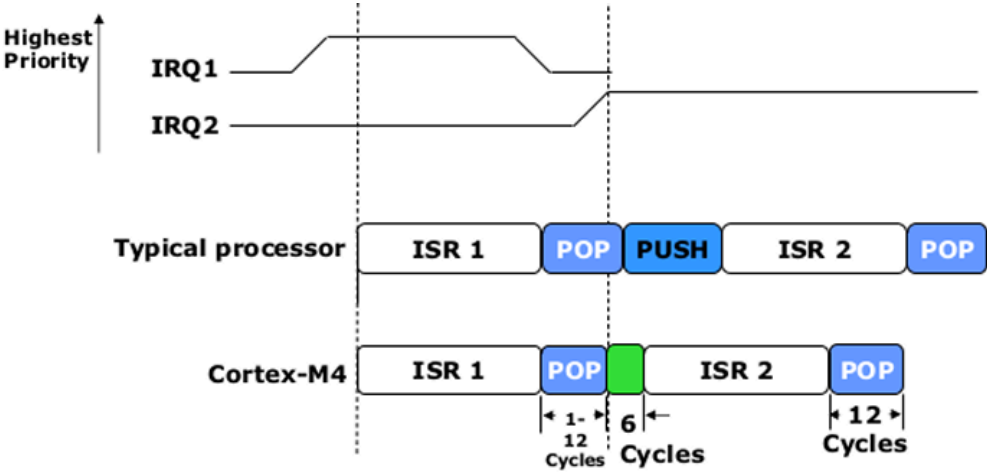


Figura 9: Interrupt Latency - Pre-Emption

1.3. Late Arrival (Llegada Tardía)

Ocurre cuando una interrupción de **alta prioridad** se dispara justo mientras el procesador está realizando la fase de apilado automático de registros (**PUSH**) para una interrupción previa de **menor prioridad**.

💡 Optimizador de Llegada Tardía

- 1. El procesador **no interrumpe ni descarta** el proceso de apilado en curso, ya que la información del contexto que debe guardarse en el Stack es idéntica independientemente de qué ISR se ejecute primero.
- 2. En paralelo a la finalización del apilado, la CPU cambia el vector buscado en la tabla IVT y **salta a ejecutar primero la ISR de mayor prioridad** (la que llegó tarde).
- 3. Al terminar la ISR de alta prioridad, el procesador atiende a la de menor prioridad mediante **Tail-Chaining** (6 ciclos).

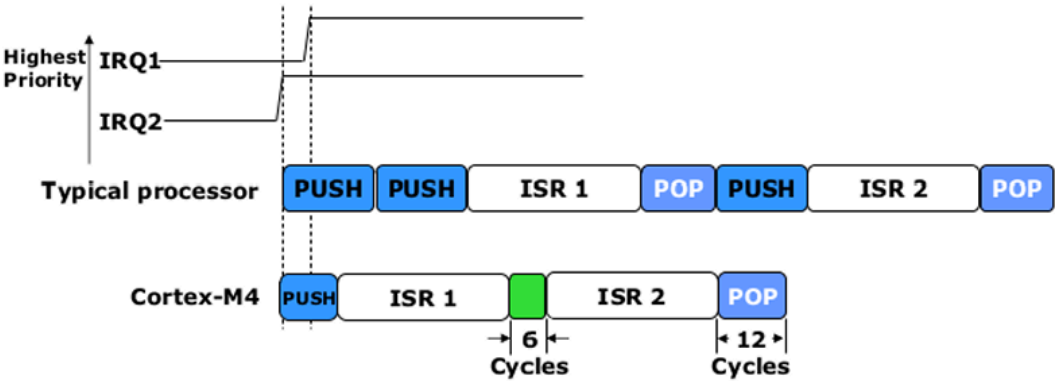


Figura 10: Interrupt Latency - Late Arrival

Mecanismo de HW	Condición de Disparo	Impacto en Latencia / Ciclos
Tail-Chaining	ISR finaliza y hay otra interrupción pendiente.	Transición directa en 6 ciclos (omite POP + PUSH completo).
Pre-Emption	Entra IRQ con prioridad estrictamente mayor.	Desaloja ISR actual; se anida mediante nuevo PUSH de contexto.
Late Arrival	Entra IRQ de alta prioridad durante el PUSH de una menor.	Aprovecha el PUSH en curso y redirige el salto a la ISR más urgente.

El Temporizador de Sistema: SysTick

El **SysTick (System Tick Timer)** es un temporizador regresivo de 24 bits integrado internamente dentro del núcleo ARM Cortex-M4 (Excepción N° 15 de la tabla IVT).

2.1. Propósito y Arquitectura

- **Heartbeat del Sistema:** Proporciona una base de tiempo fija y confiable (típicamente a **1 kHz / 1 ms**) utilizada por el sistema operativo o scheduler para medir retrasos, marcas de tiempo y coordinar tareas periódicas.
- **Contador Regresivo de 24 Bits:** Módulo decrementador independiente que cuenta desde el valor cargado en `LOAD` hasta 0.
- **Excepción del Núcleo:** Al llegar a cero, se activa el flag `COUNTFLAG` y, si está habilitado, dispara la excepción nativa `SysTick_Handler()`.

⚠ Limpieza Automática de Flag por Hardware

A diferencia de los periféricos externos de los puertos (como `PORTA` o `PORTB`), **la ISR del SysTick NO requiere limpiar un flag mediante software (w1c)**. El hardware borra automáticamente la solicitud de interrupción al ingresar a la rutina `SysTick_Handler()`.

2.2. Estructura de Registros (SysTick_Type en CMSIS)

El módulo se programa mediante cuatro registros principales agrupados en la estructura `SysTick`:

⚙ Registros del Módulo SysTick

- **SysTick->CTRL (Control and Status Register):**
 - Bit 0 (`ENABLE`): Activa o pausa el contador (1 = Encendido).
 - Bit 1 (`TICKINT`): Habilita la excepción de interrupción (1 = Genera IRQ al llegar a 0).
 - Bit 2 (`CLKSOURCE`): Selecciona la fuente de reloj (1 = Core Clock / 100 MHz; 0 = Clock externo).
 - Bit 16 (`COUNTFLAG`): Retorna 1 si el contador llegó a 0 desde la última lectura.
- **SysTick->LOAD (Reload Value Register):** Registro de 24 bits (valor máx: $2^{24} - 1$). Define el valor inicial cargado al llegar a cero.
- **SysTick->VAL (Current Value Register):** Valor actual del contador de 24 bits. Escribir cualquier valor borra el contador y limpia `COUNTFLAG`.
- **SysTick->CALIB (Calibration Value Register):** Registro de lectura con la calibración de fábrica para 10 ms.

Cálculo del Valor de Recarga (LOAD) Para obtener un intervalo de tiempo T con una frecuencia de reloj f_{core} :

$$LOAD = (f_{core} \times T) - 1$$

🔧 Ejemplos de Cálculo de Reload para K64F ($f_{core} = 100 \text{ MHz}$)

- **Para Tick de 1 ms (1 kHz):**

$$LOAD = (100.000.000 \text{ Hz} \times 0,001 \text{ s}) - 1 = 100.000 - 1 = 99.999 \text{ } 0x1869F$$

- **Para Tick de 125 ms (8 Hz):**

$$LOAD = (100.000.000 \text{ Hz} \times 0,125 \text{ s}) - 1 = 12.500.000 - 1 = 12.499.999 \text{ } 0xBEBC1F$$

2.3. Código C de Inicialización y Controlador Nativo

```
#include "MK64F12.h"

#define SYSTICK_CORE_HZ 100000000UL // 100 MHz
#define SYSTICK_FREQ_HZ 1000U      // 1 kHz (1 ms)

// 1. Inicialización del SysTick a 1 kHz
void SysTick_Init(void) {
    // Deshabilitar el temporizador antes de configurar
    SysTick->CTRL = 0x00;

    // Configurar valor de recarga para 1 ms
    SysTick->LOAD = (SYSTICK_CORE_HZ / SYSTICK_FREQ_HZ) - 1UL;

    // Limpiar el contador actual
    SysTick->VAL = 0x00;

    // Seleccionar Core Clock, habilitar interrupción y activar SysTick
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                  SysTick_CTRL_TICKINT_Msk |
                  SysTick_CTRL_ENABLE_Msk;
}

// 2. Manejador de la Interrupción (ISR nativa del núcleo)
void SysTick_Handler(void) {
    // No requiere limpiar flags por software.
    // Tareas periódicas o incremento de contador de ms:
    milisegundos_transcurridos++;
}
```

3. Patrón de Arquitectura: Despachador de Tareas Periódicas

Para evitar configurar múltiples timers de hardware para cada tarea ligera, el SysTick actúa como una **base de tiempo compartida** mediante el registro de callbacks (**Scheduler** / Fachada HAL).



Ventajas del Despachador Periódico (SysTick Multi-Callback)

1. **Consolidación:** Múltiples módulos de software (refresco de display multiplexado, lectura de encoder con antirrebote, temporizadores de software) se enganchan al mismo tick de 1 kHz.
2. **Desacoplamiento:** La ISR del SysTick no conoce la lógica interna de los drivers; simplemente recorre un vector de punteros a función y los ejecuta en orden secuencial.

Ejemplo de Despachador Genérico en C

```
#define MAX_CALLBACKS 4

typedef void (*systick_callback_t)(void);

static systick_callback_t callbacks_list[MAX_CALLBACKS] = {0};
static uint8_t callback_count = 0;

// Registrar una tarea periódica
bool SysTick_AddCallback(systick_callback_t callback) {
    if (callback_count < MAX_CALLBACKS && callback != NULL) {
```

```

        callbacks_list[callback_count++] = callback;
        return true;
    }
    return false;
}

// ISR del SysTick: desparrama la ejecución a todos los callbacks
void SysTick_Handler(void) {
    for (uint8_t i = 0; i < callback_count; i++) {
        if (callbacks_list[i] != NULL) {
            callbacks_list[i](); // Ejecuta HAL_Refresh_Task, actualizarEncoder, etc.
        }
    }
}

```

Análisis Práctico de Timing y Prioridades

En sistemas embebidos reales con múltiples interrupciones concurrentes, asignar la misma prioridad a todas las ISRs puede provocar la pérdida de eventos críticos.

Caso de Estudio: Lector de Banda Magnética vs. Refresco de Display

- **Conflicto:** La ISR de refresco de display (que hace **bit-banging** por shift-registers en SysTick) es la tarea más larga del sistema.
- **Problema:** Si el SysTick tuviera la misma prioridad que la interrupción de puerto PORTB del Lector de Tarjetas (ambos en prioridad 0 por defecto), la ejecución del SysTick retrasaría la captura del flanco del reloj de la tarjeta, perdiendo bits del **swipe**.
- **Solución:** Ajustar las prioridades mediante el NVIC bajándole la prioridad al SysTick:

```

// Asignación de Prioridades en App_Init()
void App_Init(void) {
    HAL_Scheduler_GlobalIRQDisable();

    // Configurar SysTick con prioridad 1 (más baja que la prioridad 0 por defecto de PORTB)
    NVIC_SetPriority(SysTick_IRQn, 1);

    // Inicializar el lector de tarjetas (queda en prioridad 0 por defecto)
    inicializarLector();

    HAL_Scheduler_GlobalIRQEnable();
}

```

Verificación con Osciloscopio (Pin de Testeo TP)

Mediante el uso de un pin de prueba GPIO (PIN_TEST_ISR), se conmuta a nivel HIGH al ingresar a una ISR y a LOW al salir. Un osciloscopio permite medir con precisión absoluta:

- **Duración de la ISR (T_{exec}):** Porcentaje del período del tick consumido por el firmware.
- **Latencia de Interrupción:** Tiempo exacto entre el flanco físico del periférico externo y la entrada efectiva del procesador a la rutina de servicio.

Arquitectura de Firmware en Capas

En el desarrollo de sistemas embebidos profesionales, la complejidad crece rápidamente al integrar múltiples periféricos (displays multiplexados, teclados, sensores, comunicaciones). Para evitar que el código se vuelva inmanejable ("código espagueti") y garantizar su mantenibilidad, portabilidad y testeabilidad, se utiliza una **arquitectura modular estructurada en capas de abstracción bien definidas**.

Principio de Diseño en Capas

Cada capa del sistema satisface un rol específico y se comunica **exclusivamente con la capa inmediata superior o inferior** mediante interfaces estándar. La regla fundamental es el **aislamiento de hardware**: la capa de aplicación debe desconocer por completo qué microcontrolador o circuito integrado físico está ejecutando el código.

Jerarquía de Abstracción del Sistema

Capa	Responsabilidad y Función	Impacto ante Cambios
Aplicación (APP)	Contiene la lógica de negocio (FSM, reglas del producto). Es independiente del tiempo real y nunca accede directamente a registros .	Totalmente portable. No cambia si se reemplaza el procesador o los componentes externos.
Hardware Abstraction Layer (HAL)	Abstrae los componentes de hardware externos (display de 7 segmentos, shift-registers 74HC595, lector de tarjetas, encoder).	Si cambia un componente exterior (ej. nuevo display), solo se modifica esta capa .
Microcontroller Abstraction Layer (MCAL)	Librería de más bajo nivel. Modifica los registros del microcontrolador (PORT, GPIO, SysTick, ADC, PIT, UART).	Si se cambia de microcontrolador (ej. K64F → STM32), solo se modifica esta capa .
Complex Drivers	Módulos especiales que unifican MCAL y HAL para aplicaciones de tiempo crítico estricto o máxima velocidad.	Usados solo cuando el paso por capas intermedias introduce latencia inaceptable.

Regla Inviolable de Arquitectura

La Capa de Aplicación (APP) jamás debe incluir `MK64F12.h` ni acceder a registros como `PORTB->PCR` o `PTB->PSOR`. Toda interacción con el mundo físico debe realizarse a través de las funciones de servicio de la capa HAL/MCAL.

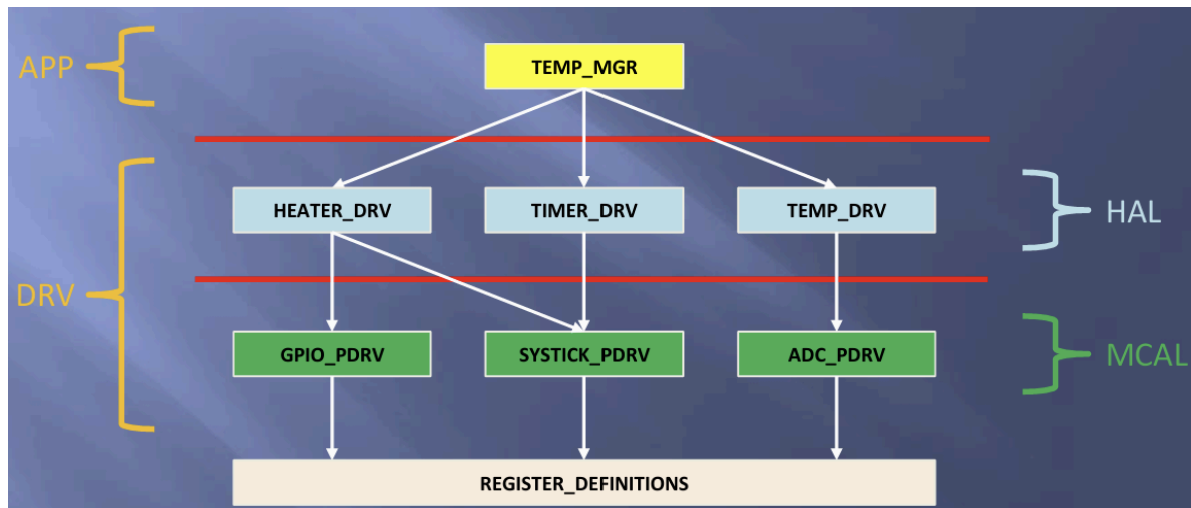


Figura 11: Ejemplo de Arquitectura de Firmware: Controlador de temperatura

Estructura Estándar de un Driver en C

Todos los drivers de periféricos (DRV) deben seguir una estructura interna uniforme dividida en 4 partes funcionales:



1. Inicialización (DRV_Init)

- Configura los registros iniciales del periférico e inicializa variables locales en RAM.
- Se ejecuta **una sola vez** al arrancar el programa (invocada desde App_Init).



2. Servicios de API (DRV_SrvN)

- Funciones públicas de interfaz con la capa superior (ej. HAL_Display_SetText).
- Invocables con cualquier frecuencia desde la APP. Ocultan los registros.



3. Rutina Periódica (DRV_PISR)

- Tarea ejecutada a frecuencia fija (ej. cada 1 ms vía callback del SysTick).
- Diseñada para **eventos lentos** (antirrebote de botones, multiplexado de display).



4. Interrupción Dedicada (DRV_ISR)

- Atiende eventos físicos instantáneos disparados por hardware.
- Diseñada para **eventos rápidos / críticos** (flancos de reloj de banda magnética).

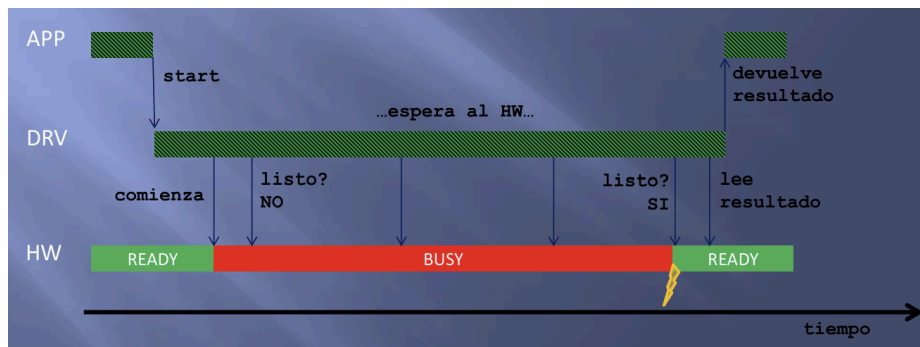
Análisis Temporal de Modelos de Ejecución

Para gestionar la interacción entre la Aplicación, el Driver y el Hardware, existen 4 estrategias temporales con eficiencias drásticamente distintas:

1. Código Bloqueante (Blocking Code)

La Aplicación llama al Driver para iniciar una tarea y el Driver congela la CPU en un bucle cerrado hasta que el Hardware responda.

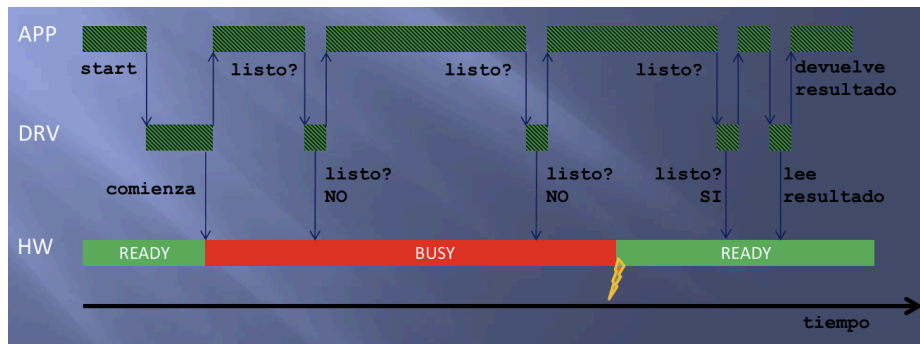
- **Rendimiento: 0% de eficiencia de CPU.** Desperdicia millones de ciclos de reloj esperando eventos externos.
- **Uso:** Totalmente **prohibido** en sistemas profesionales y de tiempo real.



2. Polling (Consulta Continua)

La Aplicación realiza una secuencia de 3 llamadas no bloqueantes: **comenzar**, **¿está listo?**, y **leer resultado**.

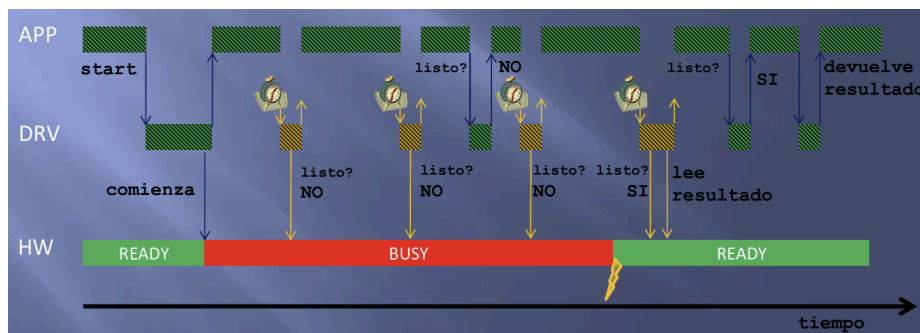
- **Rendimiento:** La APP puede intercalar otras tareas, pero pierde tiempo valioso volviendo a preguntar constantemente ¿listo? al Driver.
- **Inconveniente:** El Driver actúa de simple pasamanos y la CPU consume energía inútilmente.



3. Interrupción Periódica (PISR / Hardware Polling)

Un temporizador de hardware (como el **SysTick a 1 kHz**) ejecuta automáticamente la función `DRV_PISR()` a intervalos regulares para interrogar al hardware.

- **Rendimiento: Excelente.** La APP queda suspendida un tiempo ínfimo durante la interrupción y el evento se captura en RAM inmediatamente.
- **Uso:** Opción preferida para la mayoría de las interfaces humanas (botones, encoders, displays).



4. Interrupción Dedicada (ISR)

El hardware envía una señal eléctrica (IRQ) a la CPU en el instante exacto en que ocurre el evento, ejecutando `DRV_ISR()`.

- **Rendimiento: Respuesta inmediata en tiempo real (mínima latencia).**

- **Uso:** Reservado para eventos ultra-rápidos que se perderían entre muestras periódicas.

Modelo	Latencia	Consumo de CPU	Caso de Uso Recomendado
Bloqueante	Nula (espera dedicada)	100% de la CPU desperdiciada	Inicializaciones sencillas de hardware al arrancar.
Polling	Variable según el loop	Alto (consultas repetitivas)	Pruebas de laboratorio o microcontroladores sin timers.
PISR (Tick)	Acotada al período (ej. 1 ms)	Mínimo (ejecución rápida en ISR)	Pulsadores, encoders, displays multiplexados, teclados.
ISR Dedicada	Inmediata (≈ nanosegundos)	Solo durante la ráfaga del evento	Lectores de banda magnética, recepción UART, encoders de motor.

Integración Segura con Callbacks y Reglas de Diseño

! REGLA ABSOLUTA DE DISEÑO: Jamás Lanzar Callbacks desde ISR / PISR

“Never ever launch a callback from inside an ISR or PISR.”

Si un driver ejecuta un **callback** inyectado por el usuario directamente dentro del contexto de una interrupción y dicho código contiene un error (bucle infinito, retardo o sección bloqueante), **todo el microcontrolador se congelará irreversiblemente.**

Arquitectura Correcta de Despacho de Eventos

- Para comunicar eventos desde el hardware hacia la aplicación sin correr riesgos:
1. La **ISR / PISR** solo captura los datos crudos y enciende un **Flag en RAM**.
 2. El bucle principal (**App_Run**) consulta el estado del driver mediante funciones de servicio (**DRV_Srv**).
 3. Al detectar el **flag activo**, la **APP** invoca la función de usuario o **callback** de manera segura fuera de la interrupción.

Código Didáctico: Arquitectura Completa de Capas y Timer PISR en C

```
// =====
// CAPA MCAL / HAL: driver_timer.c (Gestión del SysTick y PISR)
// =====
#include "MK64F12.h"
#include <stdbool.h>

static volatile bool flag_evento_10ms = false;
static uint32_t contador_milis = 0;

// PISR: Ejecutada periódicamente a 1 kHz por la ISR del SysTick
void DRV_PISR_TimerTick(void) {
    contador_milis++;
    if ((contador_milis % 10) == 0) {
        flag_evento_10ms = true; // SOLO SE PONE EL FLAG, NO SE LLAMA AL CALLBACK
    }
}

// Servicio público para consultar estado desde la APP (Servicio No Bloqueante)
bool DRV_Srv_CheckEvent10ms(void) {
    if (flag_evento_10ms) {
        flag_evento_10ms = false; // Write-1-to-Clear lógico
        return true;
    }
    return false;
}

// =====
// CAPA APLICACIÓN: App.c (Súper-Loop y FSM)
// =====
void App_Run(void) {
    // Consulta no bloqueante de eventos del driver
    if (DRV_Srv_CheckEvent10ms()) {
        // Ejecución segura de la lógica de usuario / Callback
        FSM_Update(EV_TICK_10MS);
    }

    // El Súper-Loop continúa ejecutando otras tareas sin bloquearse
}
```

Resumen de la Estrategia del Proyecto

- **SysTick (1 kHz):** Se utiliza como la base de tiempo central (Scheduler) que dispara la PISR para refrescar el display y procesar el antirrebote del encoder.
- **Lector Magnético (ISR Dedicada):** Captura cada bit en tiempo real por flanco en la ISR de PORTB. La decodificación del paquete (operación costosa) se delega a App_Run() fuera de la interrupción.

Fundamentos de la Comunicación Serial Asincrónica (UART)

La **UART (Universal Asynchronous Receiver-Transmitter)** es el módulo periférico encargado de traducir datos en paralelo (bytes dentro de la CPU) a un flujo secuencial de bits que viajan por una sola línea física de transmisión (TX) y se reciben por otra de recepción (RX).

Estructura de la Trama UART Estándar

A diferencia de los protocolos síncronos (donde viaja una señal de reloj compartida), la UART es **asíncrona**: emisor y receptor acuerdan previamente una velocidad común llamada **Baud Rate** (bits por segundo, bps).

 **Anatomía de una Trama de Datos**

- **Estado Reposo (Idle):** La línea física permanece en nivel lógico **HIGH (1)**.
- **Start Bit (1 bit):** Transición de 1 a 0 (flanco de bajada) que indica al receptor el inicio inminente de un byte.
- **Bits de Datos (8 o 9 bits):** Se envían secuencialmente empezando **siempre por el bit menos significativo (LSB primero)**.
- **Bit de Paridad (Opcional):** Bit de verificación de redundancia (paridad PAR o IMPAR) para detectar corrupción en la línea.
- **Stop Bit (1 o 2 bits):** Retorno de la línea a nivel lógico **HIGH (1)** para finalizar el frame y restablecer el reposo.

IDLE	START	BITS DE DATOS (D0 .. D7)	PARIDAD	STOP
Nivel HIGH (1)	0 (1 bit)	LSB primero → MSB último (8 bits)	Opcional	Nivel HIGH (1)

Estándares Eléctricos y Conversión de Niveles

El microcontrolador Kinetis K64F opera externamente con niveles **TTL / CMOS (0V = '0', 3.3V = '1')**. Para interconectarse con computadoras u otros sistemas industriales se requieren integrados conversores:

 **Norma RS-232**

- Utiliza lógica invertida con altos voltajes:
 - '0' lógico: +3V a +15V
 - '1' lógico: -3V a -15V
- Requiere un chip transceptor (ej. MAX232) para adaptar los niveles eléctricos a los 3.3V del microcontrolador.

 **Interfaz USB-UART (OpenSDA)**

- La placa **FRDM-K64F** incluye un segundo chip microcontrolador configurado como interfaz **OpenSDA**.
- Convierte las señales RX/TX del puerto UART0 en un puerto COM virtual USB para la PC.

Cálculo y Ajuste Fino del Baud Rate en Kinetis K64F

Para lograr un muestreo preciso sin deriva temporal en comunicaciones asincrónicas, el K64F dispone de una arquitectura de división de reloj con **ajuste grueso (SBR)** y **ajuste fino (BRFA)**.

Fórmula Matemática del Baud Rate

$$\text{Baud Rate} = \frac{\text{UART_Clock}}{16 \times \left(\text{SBR} + \frac{\text{BRFA}}{32} \right)}$$

⚙️ Campos de Registros para el Baud Rate

1. Módulo Baud Rate SBR (13 bits):

- Dividido en dos registros: UARTx->BDH (bits[12:8] altos) y UARTx->BDL (bits[7:0] bajos).
- Valor entero de división gruesa: $\text{SBR} = \left\lfloor \frac{\text{UART_Clock}}{16 \times \text{Baud Rate}} \right\rfloor$.

2. Baud Rate Fine Adjust BRFA (5 bits):

- Ubicado en los bits[4:0] del registro UARTx->C4.
- Permite ajustar la parte fraccionaria multiplicando el residuo decimal por 32:

$$\text{BRFA} = \text{round} \left(\left(\left(\frac{\text{UART_Clock}}{16 \times \text{Baud Rate}} \right) - \text{SBR} \right) \times 32 \right)$$

🔧 Ejemplo de Cálculo para 115200 Baudios a 60 MHz

Si el reloj asignado al periférico UART es $f_{\text{bus}} = 60\text{MHz}$ y se requiere operar a 115200 baudios:

- Divisor Teórico = $60 \times \frac{10^6}{16 \times 115200} = 32.55208$
 $\text{SBR} = 32 \rightarrow$ Escribir 32 en BDL
- (BDH = 0)
 Parte Fraccionaria = $0.55208 \times 32 = 17.666 \rightarrow \text{BRFA} = 18$
- (Escribir 18 en C4)

3. Registros de Datos, Estado y Control del Periférico (UARTx)

En el SDK de NXP / CMSIS, la comunicación UART se controla a través de la estructura UART_Type.

Registro	Modo	Descripción y Uso Técnico
UARTx->D	R / W	Registro de Datos: Físicamente son dos buffers separados. Escribir en D coloca el byte en la cola de transmisión. Leer D retira el byte recibido de la cola de recepción.
UARTx->S1	R / w1c	Status Register 1: Contiene las banderas de estado del periférico (TDRE, TC, RDRF).
UARTx->C2	R / W	Control Register 2: Habilita el transmisor (TE), receptor (RE) e interrupciones (TIE, RIE).
UARTx->C4	R / W	Control Register 4: Contiene el ajuste fino de reloj ("BRFA") y control de la FIFO.

Banderas Clave de Estado (UARTx->S1)



1. RDRF (Bit 5)



2. TDRE (Bit 7)



3. TC (Bit 6)

Receive Data Register Full:

Se pone en 1 por hardware cuando ha ingresado un nuevo byte completo a la UART.

Limpieza: Se limpia automáticamente al leer S1 y luego leer D.

Transmit Data Register Empty:

Se pone en 1 cuando el buffer de transmisión está libre para recibir un nuevo byte.

Limpieza: Se limpia automáticamente al leer S1 y luego escribir en D.

Transmission Complete:

Se pone en 1 cuando el último bit salió físicamente del pin TX. **Uso:** Indispensable para transceptores RS-485 / RS-422 antes de cortar la línea.

4. Arquitectura de Firmware e Interrupciones en UART

Debido a que el tiempo de transmisión de un byte a 9600 baudios toma aproximadamente 1.04ms (un abismo en ciclos de CPU a 120MHz), **nunca se debe utilizar código bloqueante en sistemas de tiempo real.**

Estrategia de Interrupciones Compartidas (RX / TX Vector)

En el K64F, las interrupciones de transmisión y recepción de un mismo canal UART están **multiplexadas en un único vector de la IVT** (por ejemplo UART0_RX_TX_IRQn). Por lo tanto, la ISR debe consultar el registro S1 para identificar la causa:

Estructura Canónica de una ISR de UART

```
void UART0_RX_TX_IRQHandler(void) {
    uint8_t status = UART0->S1;

    // 1. ATENDER RECEPCIÓN (RDRF = 1)
    if (status & UART_S1_RDRF_MASK) {
        uint8_t data = UART0->D; // La lectura de D limpia el flag RDRF de forma automática
        RingBuffer_Put(&rx_queue, data);
    }

    // 2. ATENDER TRANSMISIÓN (TDRE = 1 y TIE habilitado)
    if ((status & UART_S1_TDRE_MASK) && (UART0->C2 & UART_C2_TIE_MASK)) {
        uint8_t tx_data;
        if (RingBuffer_Get(&tx_queue, &tx_data)) {
            UART0->D = tx_data; // La escritura en D limpia el flag TDRE
        } else {
            // Si no quedan datos para enviar, deshabilitar TIE para evitar un loop infinito
            UART0->C2 &= ~UART_C2_TIE_MASK;
        }
    }
}
```

Regla Crítica: Habilitación Dinámica de TIE

A diferencia de la interrupción de recepción (RIE), la interrupción por buffer libre (TIE) **nunca debe dejarse habilitada de forma permanente**. Dado que el registro D permanece vacío la mayor parte del tiempo, si TIE se mantiene activo, la CPU caerá en un **bucle infinito de interrupciones de transmisión**.

Mecanismo Correcto: TIE solo se activa por software en la función de envío cuando se encola un nuevo dato, y la propia ISR se encarga de apagar TIE cuando la cola de transmisión queda vacía.

5. Colas de Hardware (FIFOs) y Configuración de Watermark

Para aplicaciones con tráfico de datos intensivo a altas velocidades (ej. 1Mbps), la arquitectura del Kinetis K64F incluye **FIFOs de hardware** independientes para TX y RX de hasta 8 bytes de capacidad.

Concepto de Marca de Agua (Watermark)

La marca de agua (**Watermark**) permite programar el umbral exacto de llenado o vaciado de la FIFO que disparará la interrupción, reduciendo drásticamente la cantidad de llamadas a la ISR:



Watermark en Recepción (RWFIFO)

- Determina cuántos bytes deben acumularse en la FIFO de recepción antes de generar la IRQ.
- **Ejemplo:** Si $RWFIFO = 4$, la CPU solo es interrumpida cuando la FIFO ha almacenado 4 bytes, procesándolos todos en una sola entrada a la ISR.



Watermark en Transmisión (TWFIFO)

- Determina cuántas posiciones libres deben quedar en la FIFO de transmisión para solicitar recarga.
- **Ejemplo:** Permite a la ISR rellenar hasta 8 bytes consecutivos de un tiro mediante una ráfaga (**burst**).



Habilitación de FIFOs en UARTx->PFIFO

- **TXFE / RXFE (Bits 7 y 3):** Habilitan la FIFO de transmisión y recepción respectivamente.
- **TXFIFOSIZE / RXFIFOSIZE:** Informan la capacidad física de hardware soportada por el canal de la UART (2^{SIZE} bytes).